Reinforcement Learning with Function Approximation

Justus Piater

January 9, 2025

These notes loosely follow The Book and its notation [Sutton and Barto 2018].

Contents

1 State and Action Value Function Approximation **1**

2 Policy-Gradient Methods **9**

3 Implementation and Experiments **17**

4 Actor-Critic Methods **20**

5 Case Studies of Off-Policy Actor-Critic Algorithms **23**

6 Offline Reinforcement Learning **30**

7 On-Policy Reinforcement Learning **31**

8 Summary **34**

1 State and Action Value Function Approximation

Limitations of tabular methods? [Slide 1]

Limitations of tabular representations:

- time required to fill large tables
- space occupied by large tables
- quantization errors
- no generalization

Continuous-State RL [Slide 2]

Continuous state space, discrete actions: Approximate $q(s,a)$ by a set of differentiable functions $\hat{q}_a(s,\mathbf{w})$ of the state vector s that are parametrized by a vector $\mathbf{w}$.

Action selection: as before.

Value-Function Update:

$$\begin{aligned} Q(s,a) &\leftarrow Q(s,a) + \alpha\left[u - Q(s,a)\right] \\ \hat{q}_a(s,\mathbf{w}) &\overset{?}{\leftarrow} \hat{q}_a(s,\mathbf{w}) + \alpha\left[u - \hat{q}_a(s,\mathbf{w})\right] \end{aligned}$$

Value-Function Update by Stochastic Gradient Descent [Slide 3]

Having observed state s and update target u, adjust $\mathbf{w}$ by a step in the direction that most reduces the squared error:

$$\begin{aligned} \mathbf{w} &\leftarrow \mathbf{w} - \frac{1}{2}\alpha\nabla_{\mathbf{w}}\left[u - \hat{q}_a(s,\mathbf{w})\right]^2 \\ &= \mathbf{w} + \alpha\left[u - \hat{q}_a(s,\mathbf{w})\right]\nabla_{\mathbf{w}}\hat{q}_a(s,\mathbf{w}) \\ \nabla_{\mathbf{w}}\hat{q}_a(s,\mathbf{w}) &= \left(\frac{\partial\hat{q}_a(s,\mathbf{w})}{\partial w_1}, \frac{\partial\hat{q}_a(s,\mathbf{w})}{\partial w_2}, \dots, \frac{\partial\hat{q}_a(s,\mathbf{w})}{\partial w_n}\right)^{\mathrm{T}} \end{aligned}$$

To understand this kind of gradient descent, consider e.g. a proportional function:

$$\begin{aligned} & & \hat{q}(s,w) &= ws \\ & & w &\leftarrow w + \alpha\left(u - ws\right)s \\ \text{For } w=1,\ s=3,\ u=6: & & w &\leftarrow 1 + \alpha(6-3)\cdot 3 &&= 1 + 9\alpha \\ \text{For } \alpha = \frac{1}{9}: & & w &\leftarrow 1 + 9\cdot\frac{1}{9} &&= 2 \\ & & \hat{q}(3,2) &= 2\cdot 3 &&= 6 \end{aligned}$$

What do we lose with respect to tabular methods?

- Due to gradient descent, we typically have to settle for a *local* optimum.
- The limited expressivity of the parametric q function restricts the scope of learnable value functions, and thus policies.
- Since the q function is approximate, convergence guarantees – so they exist – are generally weaker.

As before, u can be obtained in various fashions, including Monte Carlo ($u = G_t$), Sarsa ($u = r + \gamma\hat{q}_{a'}(s',\mathbf{w})$) or Q-Learning ($u = r + \gamma\max_{a'}\hat{q}_{a'}(s',\mathbf{w})$).

Note

Bootstrapped targets u implicitly depend on $\mathbf{w}$, but this is ignored in the above gradient computation. Thus, this SGD method gives rise to so-called ***semi-gradient*** Sarsa, Q-Learning, etc.

[Sutton and Barto 2018 Sec. 9.3]

Linear Function Approximation [Slide 4]

Using a vector-valued ***feature function*** $\phi(s)$ that maps a state s to its feature vector:

$$\hat{q}(s, \mathbf{w}) = \sum_{i=1}^{n} w_i \phi_i(s)$$
$$\nabla_{\mathbf{w}} \hat{q}(s, \mathbf{w}) = \phi(s)$$

Example: ***Continuous-state*** (x, y) ***maze with unit-step N/E/S/W actions***

Would a linear function approximator $\hat{q}_a(s, \mathbf{w}) = w_0 + w_1 x(s) + w_2 y(s)$ be appropriate?

Hardly, since this function defines a plane over the state space, whereas optimal actions

- aim for the target state(s) from all directions,
- vary locally depending on obstacles.

Q: Can we choose better feature functions for this scenario?

A: Yes, since you can approximate any function with an appropriate choice of feature functions.

[Sutton and Barto 2018 Sec. 9.4]

Continuing Tasks: Discounting with Function Approximation? [Slide 5]

- Feature functions and approximation alias states. Consider the extreme case of all states appearing the same.
- Then, to evaluate the performance $J(\pi)$, all we have is the reward sequence.
- Since the state sequence has no beginning and no end, we might as well look at the *average reward* $r(\pi)$ at each time step.
- Then, since all states are equivalent, the expected discounted return over an infinite time horizon is the geometric series $J(\pi) = r(\pi) + \gamma r(\pi) + \gamma^2 r(\pi) + \cdots = r(\pi)/(1-\gamma)$.
- Thus, all such returns for different γ are proportional, and thus the performance ranking of different policies does not depend on γ, and thus discounting is useless!

  This result taints RL on arbitrary MDPs.

In tabular RL, discounting can nevertheless be helpful for learning thanks to the policy improvement theorem: Changing the policy to improve the (discounted) value of a given state is guaranteed to improve the overall policy. However, with function approximation, due to state aliasing, we have no such theorem.

[Sutton and Barto 2018 Sec. 10.4]

Average-Reward RL [Slide 6]

$$
\begin{aligned}
r(\pi) &= \lim_{h\to\infty} \frac{1}{h} \sum_{t=1}^{h} \mathbb{E}[R_t \mid S_0, A_{0:t-1} \sim \pi] \\
&= \lim_{t\to\infty} \mathbb{E}[R_t \mid S_0, A_{0:t-1} \sim \pi] \\
&= \sum_s \mu_\pi(s) \sum_a \pi(a|s) \sum_{s',r} p(s', r \mid s, a) r \\
\mu_\pi(s) &= \lim_{t\to\infty} p(S_t = s \mid A_{0:t-1} \sim \pi)
\end{aligned}
$$

$\mu_\pi(s)$ is the ***steady-state state distribution*** for π, which is assumed to exist and to be independent of S_0 (*ergodicity*).

With some minor technicalities we can define an analogous steady-state *state density function* $\mu_\pi(s)$, $s \in \mathcal{S}$, for continous state spaces $\mathcal{S}$.

With the following definitions, many theoretical results and most algorithms carry through unchanged:

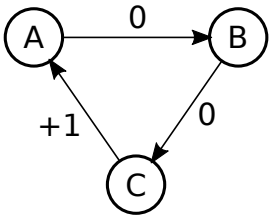
$$
\begin{aligned}
G_t &= R_{t+1} - r(\pi) + R_{t+2} - r(\pi) + \cdots && \text{differential return} \\
\delta_t &= R_{t+1} - \bar{R}_t + \hat{v}(S_{t+1}, \mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_t) && \text{differential TD error} \\
\delta_t &= R_{t+1} - \bar{R}_t + \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t) - \hat{q}(S_t, A_t, \mathbf{w}_t) && \text{differential TD error} \\
\mathbf{w}_{t+1} &= \mathbf{w}_t + \alpha \delta_t \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w}_t) && \text{similarly for } \hat{v}(S_{t+1}, \mathbf{w}_t) \\
\bar{R}_t &= \cdots && \text{estimated average reward}
\end{aligned}
$$

[Sutton and Barto 2018 Sec. 10.3]

Quick-Think: Estimating the Average Reward $\bar{R}$ [Slide 7]

This exercise provides good intuition for average-reward RL!

Consider this Markov Reward Process. What is the true value of $\bar{R}$?



A solution is available.

To estimate $\bar{R} \leftarrow \bar{R} + \beta\rho$ incrementally as usual:

1. First idea: $\rho = R - \bar{R}$.

   Holding $\bar{R}$ fixed at its true value, what is the sequence of ρ errors?

2. Better idea: $\rho = \delta = R - \bar{R} + \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$.

   Upon convergence, what are the values $\hat{v}(\cdot, \mathbf{w})$ of the three states?

   A solution is available.

   Holding $\bar{R}$ fixed at its true value, what is the sequence of δ errors?

A solution is available.

Updating $\bar{R}$ towards the differential TD error δ rather than towards the differential reward *reduces the variance* in the updates and can thus *accelerate learning*!

Intuition:

- $R - \bar{R}$ will always fluctuate (unless $R = \bar{R}$ for all transitions), while δ can converge to zero for all transitions (depending on the stochasticity of the agent and its environment).
- Immediate rewards give rise to the value differences $v(S) - v(S')$ between sucessive states. High rewards incur large such differences, and low rewards incur small such differences. Thus, subtracting this difference from the immediate reward tends to cancel this effect, reducing variance. (In this sense, $v(S) - v(S')$ serves as a ***baseline*** for estimating $\bar{R}$, a notion that we will shortly introduce formally.)
- The difference $v(S) - v(S')$ is a bootstrap estimate of the immediate (differential) reward. Just as bootstrapping helps in estimating value functions, it helps in estimating the average reward.

[Sutton and Barto 2018 Exercise 10.8]

Quick-Think [Slide 8]

Write down the update rules for the following RL algorithms:

1. episodic, semi-gradient Sarsa
2. average-reward, semi-gradient Sarsa
3. episodic, semi-gradient Q-Learning
4. average-reward, semi-gradient Q-Learning

Note

Average-reward and *differential* are often used interchangeably.

A solution is available.

[Sutton and Barto 2018 Eqn. 11.5]

Differential Semi-Gradient Sarsa [Slide 9]

1. Initialize $\mathbf{w}$, e.g. to $\mathbf{0}$.
   Initialize $\bar{R}$, e.g. to 0.
   Initialize S, the starting state.
2. Loop forever:
   (a) Take action A, observe R and S'.
   (b) Choose A' according to π as a function of $\hat{q}(S', \cdot, \mathbf{w})$.
   (c) Update rule 2 from above, followed by

$$\begin{array}{rcl} \bar{R} & \leftarrow & \bar{R} + \beta\delta \\ S & \leftarrow & S' \\ A & \leftarrow & A' \end{array}$$

Performance Measure $J(\pi)$ [Slide 10]

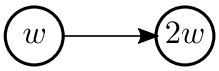
We informally introduced the performance measure $J(\pi)$ of a policy π.
Example definitions of $J(\pi)$:

- $v_\pi(s_0) = \mathbb{E}_\pi[G_0]$ for episodic tasks
- $r(\pi)$ for continuing, average-reward tasks

Quick-Think [Slide 11]

Consider the following MDP fragment:

- Two states of values w and $2w$ respectively (simple linear function approximator)
- Unique action in first state deterministically transitions to second state under zero reward
- $\gamma = 0.9$, $\alpha = 0.1$, $w_0 = 10$



What happens if we apply the following TD(0) update rule for multiple left-to-right (off-policy) transitions:

$$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, w_t) - \hat{v}(S_t, w_t)$$
$$w_{t+1} = w_t + \alpha \delta_t \frac{\mathrm{d}}{\mathrm{d}w} \hat{v}(S_t, w_t)$$

A solution is available.

This problem of ***off-policy divergence*** can and does realistically occur in real MDPs!

[Sutton and Barto 2018 Sec. 11.2]

The Deadly Triad [Slide 12]

Instability and divergence are likely to arise whenever all of the following are present:

- Function Approximation
- Bootstrapping
- Off-Policy Training

Can we give up any of these three?

The problem is *not* due to

- control – we showed that it occurs with prediction.
- uncertainties in the environment – it also occurs with dynamic programming where the MDP is known.

[Sutton and Barto 2018 Sec. 11.3]

Case Study: Deep Q Networks (DQN) [Slide 13]

Performance breakthrough on Atari 2600 games [Mnih et al. 2015]

- Q function: one deep convnet with a separate output unit for each action
- state representation combines length-4 history of states
- SGD training with size-32 minibatches
- ***experience replay***
- ***frozen target Q function***

  In the Q update, the bootstrap target is computed using a separate $\hat{q}(s, a, \mathbf{w}')$ function that is updated as $\mathbf{w}' \leftarrow \mathbf{w}$ only once every C steps.

  This counteracts the diverging Q problem and thus breaks The Deadly Triad.

  (A related strategy is ***double Q learning*** [Sutton and Barto 2018 Sec. 6.7].)

Experience Replay [Slide 14]

Problems with online training of NN Q functions:

- In online training, successive training examples are correlated, increasing variance in the updates and causing forgetting effects.
- In on-policy training, each action taken determines the next training instance, resulting in feedback loops that may lead into local optima or to divergence.
- Each training instance is used only once, contributing only to a tiny, α-size parameter update.

Solution: Instead of training on online experience directly, store each experience (S, A, R, S') in a pool. Train *off-policy* on minibatches drawn randomly from this pool.

This breaks both sequential correlation and feedback loops, and increases data efficiency by using each training instance multple times.

Experience Replay is an essential ingredient of all Deep RL methods.

2 Policy-Gradient Methods

Action values are a fairly indirect way of learning policies, and they are not a good fit for continuous action spaces.

Can we learn policies as probability distributions over state-action pairs that do not require value functions?

Policy-Gradient Methods [Slide 15]

$$\begin{aligned} \pi(a|s,\boldsymbol{\theta}) &= p(A_t = a \mid S_t = s, \boldsymbol{\theta}_t = \boldsymbol{\theta}) && \text{parametrized policy} \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t + \alpha \widehat{\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t)} && \text{stochastic gradient ascent on } J \end{aligned}$$

Note

- Actions may be discrete or continuous.
- Action selection does not use a value function.
- Policy-gradient methods that use a value function to provide bootstrap estimates of expected returns are called ***actor-critic*** methods.

  The *actor* is the learned policy, and the *critic* is the learned value function, typically a state-value function.
- Here and through Section Finite-Difference Policy Gradient Estimator, *policy gradient* is a misnomer; the term refers to the gradient of the *objective function* J in the policy parameters.

Soft-Max in Action Preferences [Slide 16]

Continuous parameterization of a discrete action set in terms of *action preferences* $h(s, a, \boldsymbol{\theta})$, and the policy as a soft-max distribution:

$$\pi(a|s, \boldsymbol{\theta}) = \frac{\exp(h(s, a, \boldsymbol{\theta}))}{\sum_b \exp(h(s, b, \boldsymbol{\theta}))}$$

Note

Action preferences $\neq$ action values since $\hat{q}(s, a, \mathbf{w}) = \hat{\mathbb{E}}[G]$ whereas $h(s, a, \boldsymbol{\theta})$ is unrestricted.

Advantages over value-based action selection:

- Stochastic policies can approach determinism as experience accumulates, in contrast to ε-greedy or soft-max action selection in action values.
- The policy modulates its own stochasticity; no need to choose ε.
- Optimal policies may be inherently stochastic. Greedy policies in action values can only be – uniformly – stochastic if multiple actions share the maximal action value.
- Often a policy is easier to learn than a value function (which contains much more information than needed for greedy action selection).
- Often a policy can be parametrized according to prior knowledge.

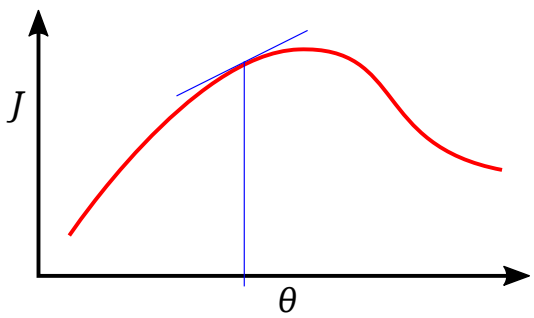
[Sutton and Barto 2018 Sec. 13.1]

Policy-Gradient RL: Basic Idea [Slide 17]

Climb up the performance function J along its gradient in the policy parameters $\boldsymbol{\theta}$:

1. Initialize $\boldsymbol{\theta}_0$
2. Repeat for each update m:
   (a) Estimate local policy gradient $\nabla_{\boldsymbol{\theta}} J_m(\boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_m}$
   (b) Update $\boldsymbol{\theta}_{m+1} = \boldsymbol{\theta}_m + \alpha_m \nabla_{\boldsymbol{\theta}} J_m(\boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_m}$

   until $\boldsymbol{\theta}_{m+1} \approx \boldsymbol{\theta}_m$ converges



Finite-Difference Policy Gradient Estimator [Slide 18]

1. Repeat for each epoch k:

   (a) Generate policy variation $\Delta\boldsymbol{\theta}_k$

   (b) Estimate $J_k(\boldsymbol{\theta}_m + \Delta\boldsymbol{\theta}_k) \approx \sum_{t=0}^{T} \gamma^t R_{t+1}$ from one or more roll-outs

   (c) Estimate gradient $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})\,|_{\boldsymbol{\theta}=\boldsymbol{\theta}_m}$ via regression over $(\boldsymbol{\theta}_m + \Delta\boldsymbol{\theta}_0, J_0(\boldsymbol{\theta}_m + \Delta\boldsymbol{\theta}_0)), \dots, (\boldsymbol{\theta}_m + \Delta\boldsymbol{\theta}_k, J_k(\boldsymbol{\theta}_m + \Delta\boldsymbol{\theta}_k))$

   until $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})\,|_{\boldsymbol{\theta}=\boldsymbol{\theta}_m}$ converges

In what scenarios can this method be used?

A solution is available.

The Policy Gradient Theorem [Slide 19]

$J(\boldsymbol{\theta}) = v_{\pi_{\boldsymbol{\theta}}}(s_0)$ value of starting state under $\pi_{\boldsymbol{\theta}}$

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \propto \int_{\mathcal{S}} \mu_\pi(s) \sum_a q_\pi(s,a) \nabla_{\boldsymbol{\theta}} \pi(a|s,\boldsymbol{\theta})\,\mathrm{d}s$$

We can avoid estimating $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ by varying the policy parameters and can instead estimate it using the *analytical* policy gradient and samples from $\mu_\pi(s)$ (i.e., from *real* experience)!

The Policy Gradient Theorem holds for episodic tasks and for continuing tasks under the average-reward formulation.

[Sutton and Barto 2018 Sec. 13.2]

REINFORCE: Monte Carlo Policy Gradient [Slide 20]

$$
\begin{aligned}
&\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \\
&\propto \int_{\mathcal{S}} \mu_\pi(s) \sum_a q_\pi(s,a) \nabla_{\boldsymbol{\theta}} \pi(a|s,\boldsymbol{\theta}) \,\mathrm{d}s && \text{policy gradient theorem} \\
&= \mathbb{E}_\pi \left[\sum_a q_\pi(S_t,a) \nabla_{\boldsymbol{\theta}} \pi(a|S_t,\boldsymbol{\theta}) \right] && \text{states } S_t \text{ as they arise under } \pi \\
&= \mathbb{E}_\pi \left[\sum_a \pi(a|S_t,\boldsymbol{\theta}) q_\pi(S_t,a) \frac{\nabla_{\boldsymbol{\theta}} \pi(a|S_t,\boldsymbol{\theta})}{\pi(a|S_t,\boldsymbol{\theta})} \right] && \text{rephrase as expectation, and} \\
&= \mathbb{E}_\pi \left[q_\pi(S_t,A_t) \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t|S_t,\boldsymbol{\theta})}{\pi(A_t|S_t,\boldsymbol{\theta})} \right] && \text{approx. it by sampling } A_t \sim \pi \\
&= \mathbb{E}_\pi \left[G_t \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t|S_t,\boldsymbol{\theta})}{\pi(A_t|S_t,\boldsymbol{\theta})} \right] && \text{since } q_\pi(S_t,A_t) = \mathbb{E}_\pi[G_t] \\
\boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t + \alpha G_t \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t|S_t,\boldsymbol{\theta}_t)}{\pi(A_t|S_t,\boldsymbol{\theta}_t)} && \text{REINFORCE update} \\
&= \boldsymbol{\theta}_t + \alpha G_t \nabla_{\boldsymbol{\theta}} \log \pi(A_t|S_t,\boldsymbol{\theta}_t)
\end{aligned}
$$

Note

This update rule has intuitive appeal:

- The update is proportional to the observed return.
- The update is in the direction that most increases the probability of choosing action A_t taken at time t.
- The update is inversely proportional to the current probability of taking this action; this keeps rarely-selected actions a chance and prevents frequently-selected actions from dominating even though their returns may not be great.

Why is this a Monte-Carlo method?

[Sutton and Barto 2018 Sec. 13.3]

REINFORCE: The Algorithm [Slide 21]

1. Initialize $\boldsymbol{\theta}$, e.g. to $\mathbf{0}$.
2. Loop forever:
   (a) Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$ following $\pi(\cdot|\cdot,\boldsymbol{\theta})$.
   (b) For $t = 0, 1, \dots, T-1$:

$$
\begin{aligned}
G &\leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k \\
\boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla_{\boldsymbol{\theta}} \log \pi(A_t|S_t,\boldsymbol{\theta})
\end{aligned}
$$

REINFORCE: A Reversed Algorithm [Slide 22]

1. Initialize $\boldsymbol{\theta}$, e.g. to $\mathbf{0}$.
2. Loop forever:
   (a) Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$ following $\pi(\cdot|\cdot, \boldsymbol{\theta})$.
   (b) $G \leftarrow 0$
   (c) For $t = T-1, T-2, \dots, 0$:

$$G \leftarrow \gamma G + R_{t+1}$$
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla_{\boldsymbol{\theta}} \log \pi(A_t | S_t, \boldsymbol{\theta})$$

- Avoiding the summation inside the loop reduces the computational complexity of processing one episode from $O(T^2)$ to $O(T)$.
- This can also be achieved without reversal, by successively dividing by γ and subtracting from the total return G_0, at the cost of slightly more code.
- If $\pi(\cdot|\cdot, \boldsymbol{\theta})$ is a neural network, variance can be reduced by summing the gradients over all steps of an episode and then updating the weights, only once per episode. In this case, the reversal is totally inconsequential.

Quick-Think: Soft-Max in Action Preferences [Slide 23]

$\pi(a|s, \boldsymbol{\theta}) = \dfrac{\exp(h(s, a, \boldsymbol{\theta}))}{\sum_b \exp(h(s, b, \boldsymbol{\theta}))}$ soft-max policy parameterization

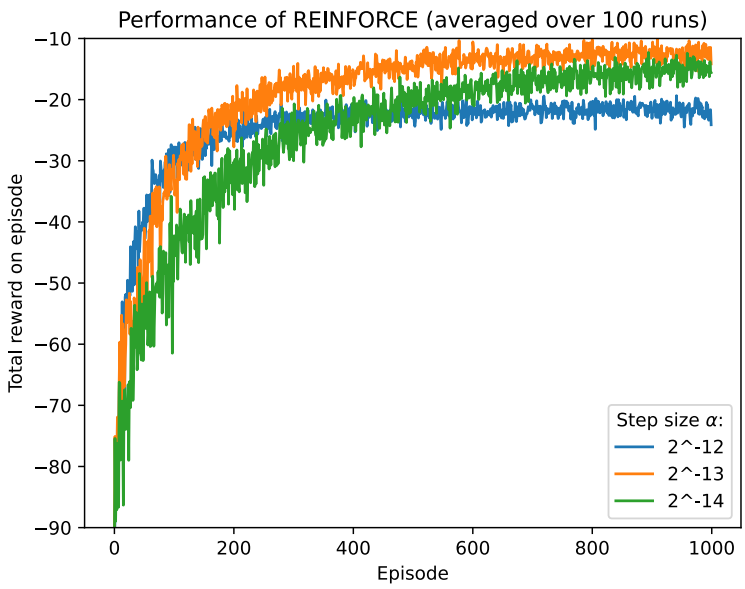
$h(s, a, \boldsymbol{\theta}) = \boldsymbol{\theta}^{\mathrm{T}} \phi(s, a)$ linear function approximator

What is the eligibility vector $\nabla_{\boldsymbol{\theta}} \log \pi(a|s, \boldsymbol{\theta})$?

A solution is available.

[Sutton and Barto 2018 Exercise 13.3]

Sensitive to learning rate... [Slide 24]



[Figure 13.1 from Sutton and Barto 2018 recreated using the authors' code]

REINFORCE with Baseline [Slide 25]

The return G_t can have high variance because it depends on the action values of S_t.

We want to emphasize the distinctions between all-valuable actions in one state, and also between all-worthless actions in another state.

Thus, use $b(s) = \hat{v}(S_t, \mathbf{w})$ as a ***baseline***.

Policy Gradient Theorem and REINFORCE update rule with baseline:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \propto \int_{\mathcal{S}} \mu_\pi(s) \sum_a \left(q_\pi(s,a) - b(s)\right) \nabla_{\boldsymbol{\theta}} \pi(a|s,\boldsymbol{\theta}) \, \mathrm{d}s$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha \left(G_t - b(S_t)\right) \nabla_{\boldsymbol{\theta}} \log \pi(A_t|S_t,\boldsymbol{\theta}_t)$$

The baseline must not depend on the action a but can otherwise be arbitrary (even random) without affecting the result:

$$\sum_a b(s) \nabla_{\boldsymbol{\theta}} \pi(a|s,\boldsymbol{\theta}) = b(s) \nabla_{\boldsymbol{\theta}} \sum_a \pi(a|s,\boldsymbol{\theta}) = b(s) \nabla_{\boldsymbol{\theta}} 1 = 0$$

Of course, the choice of $b(s)$ does affect the *estimation process* of the result; this is the whole point (here, to reduce variance in the updates).

[Sutton and Barto 2018 Sec. 13.4]

Note

$A_\pi (= q_\pi(s,a) - v_\pi(s)$ is known as the ***advantage*** function.

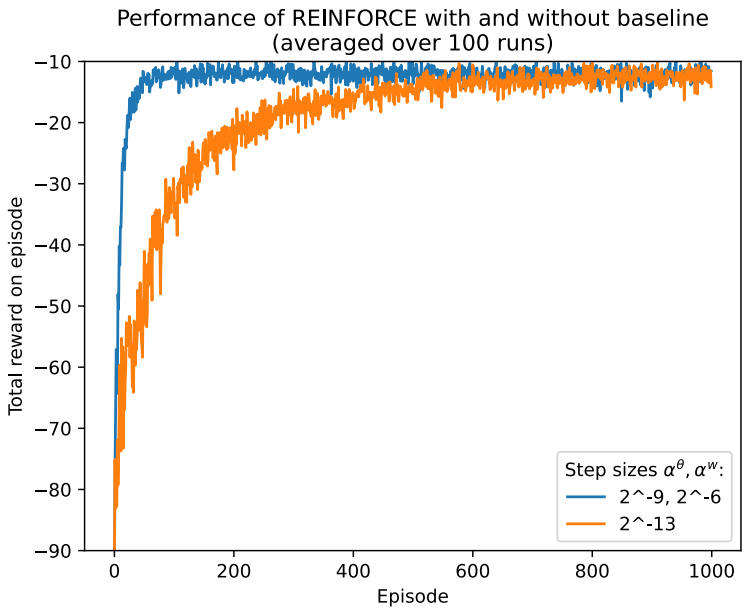
Being in state s, the (expected) advantage of taking action a rather than $a' \sim \pi(\cdot|s)$.

REINFORCE with Baseline: The Algorithm [Slide 26]

1. Initialize $\boldsymbol{\theta}$ and state-value weights $\mathbf{w}$, e.g. to $\mathbf{0}$.
2. Loop forever:
   (a) Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$ following $\pi(\cdot|\cdot, \boldsymbol{\theta})$.
   (b) For $t = 0, 1, \ldots, T-1$:

$$
\begin{aligned}
G &\leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k \\
\delta &\leftarrow G - \hat{v}(S_t, \mathbf{w}) \\
\mathbf{w} &\leftarrow \mathbf{w} + \alpha_{\mathbf{w}} \delta \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}) \\
\boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \alpha_{\boldsymbol{\theta}} \gamma^t \delta \nabla_{\boldsymbol{\theta}} \log \pi(A_t | S_t, \boldsymbol{\theta})
\end{aligned}
$$

Improved Learning Speed and Stability [Slide 27]



[Figure 13.2 from Sutton and Barto 2018 recreated using the authors' code]

Continuous Actions [Slide 28]

Policy-gradient methods require

- a differentiable policy representation
- that can be sampled from (and perhaps maximized).

Note

There is nothing in REINFORCE that even notices whether the action space is discrete or continuous (or what its dimensionality is).

We have an example for a discrete action set: Soft-Max in Action Preferences.

How might we parametrize a continuous-valued policy?

Policy as Normal Distribution [Slide 29]

Idea similar to variational autoencoders:

$$\pi(a|s,\boldsymbol{\theta}) = \frac{1}{\sigma(s,\boldsymbol{\theta})\sqrt{2\pi}} \exp\left(-\frac{(a-\mu(s,\boldsymbol{\theta}))^2}{2\sigma(s,\boldsymbol{\theta})^2}\right)$$

$\mu(s,\boldsymbol{\theta})$ and $\sigma(s,\boldsymbol{\theta})$ might e.g. be linear functions

$$\begin{aligned}
\mu(s,\boldsymbol{\theta}) &= \boldsymbol{\theta}_\mu^{\mathrm{T}}\phi_\mu(s) \\
\sigma(s,\boldsymbol{\theta}) &= \exp\left(\boldsymbol{\theta}_\sigma^{\mathrm{T}}\phi_\sigma(s)\right) \\
\boldsymbol{\theta} &= \begin{bmatrix} \boldsymbol{\theta}_\mu \\ \boldsymbol{\theta}_\sigma \end{bmatrix}
\end{aligned}$$

or (output units of) a neural network.

Quick-Think: What is the eligibility vector $\nabla_{\boldsymbol{\theta}} \log \pi(a|s,\boldsymbol{\theta})$?

Hint: Separately calculate $\dfrac{\nabla_{\boldsymbol{\theta}_\mu}\pi(a|s,\boldsymbol{\theta})}{\pi(a|s,\boldsymbol{\theta})}$ and $\dfrac{\nabla_{\boldsymbol{\theta}_\sigma}\pi(a|s,\boldsymbol{\theta})}{\pi(a|s,\boldsymbol{\theta})}$.

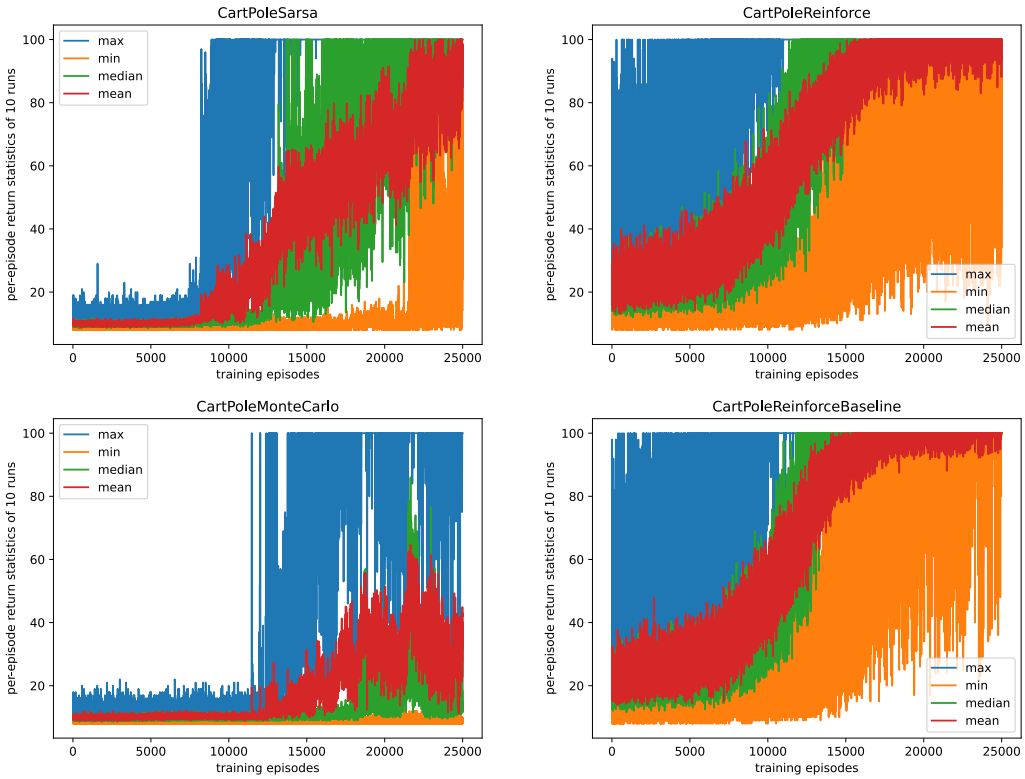
A solution is available.

[Sutton and Barto 2018 Exercise 13.4]

3 Implementation and Experiments

[Slide 30]

Value Functions; ε-greedy | **Policy Gradients; softmax in action prefs**



Parameters:

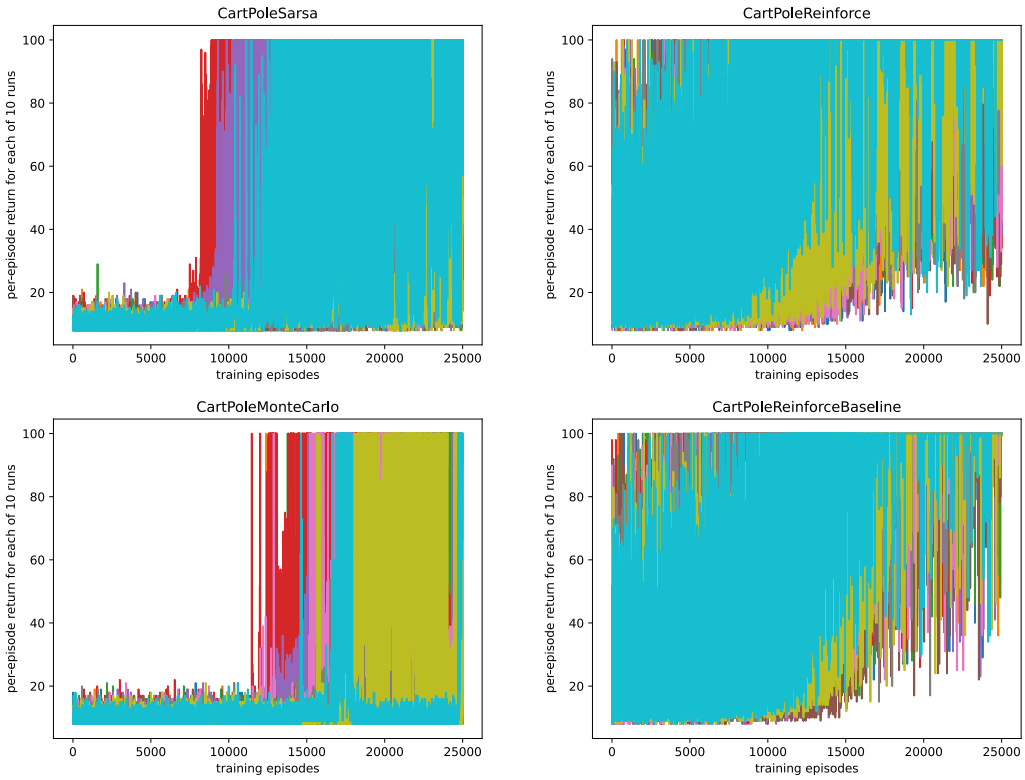
all:	γ	$=$	1			
Sarsa, Monte Carlo:	α	$=$	0.0001	ϵ	$=$	0.1
REINFORCE:	α	$=$	0.00001			
REINFORCE with Baseline:	$\alpha_{\boldsymbol{\theta}}$	$=$	0.00001	$\alpha_{\mathbf{w}}$	$=$	0.0001

Caveats:

- These are training runs with exploration, not evaluation runs using a greedy policy.
- The meta-parameters are largely untuned.
- I am confident but not 100% sure there are no significant bugs.

[Slide 31]

Value Functions; ε-greedy | **Policy Gradients; softmax in action prefs**



Other Tasks [Slide 32]

1. None of the methods are able to solve Mountain Car or Pendulum. Why not?
2. What about tasks with continuous action spaces?
3. What about *continuing* tasks?

Implementation with PyTorch [Slide 33]

- How to compute the parameter update for value-function methods?

  Can and should we use the conventional pattern:

  ```
  pred = model(train_input)
  loss = loss_fn(pred, train_output)
  optimizer.zero_grad()
  loss.backward()
  optimizer.step()
  ```

  In principle we can reformulate the value-function update as a squared-difference loss, but see a more direct way, also used for REINFORCE below.

- How to compute the parameter update for REINFORCE?

  It is unclear how (and unnecessary) to reformulate the REINFORCE update in terms of any PyTorch loss function. We can (and should) compute the gradient directly:

  In general, to compute $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha c \nabla_{\boldsymbol{\theta}} f$:

  ```
  optimizer.zero_grad()
  (-c.detach() * f).backward()
  optimizer.step()
  ```

  - If the `torch.Tensor` on which `backward()` is called contains `torch.Tensors` `c` that should not be updated, discard their gradients by way of `c.detach()` (unless you are sure that `requires_grad=False` for all of them).
  - For a soft-max policy, spare PyTorch the computation of $\log \exp(\cdot)$ and use `torch.logsumexp()`.

- We may be able to reduce variance by summing gradients over multiple time steps (say, all of an episode) and then updating according to the gradient sum. The `backward()` method accumulates gradients, so this is done by placing all `backward()` calls of an episode between `optimizer.zero_grad()` and `optimizer.step()`.

  Cf. REINFORCE: A Reversed Algorithm.

- Carefully debug every step of your algorithm.

ML methods are nasty. An implementation may work even if it is incorrect (sometimes even better than a correct implementation), or it may not work well even though it is correct.

- Make sure a $\hat{q}_a(s, \mathbf{w})$ prediction is actually closer to its target after a weight update.
- Make sure the weight updates affect exactly the intended elements, no more (e.g., constant terms) and no less (e.g., the denominator of the soft-max policy).
- To reset an `nn.Module` to a previous state, save a (detached) copy of that state and reload it later:

```
import copy
state = copy.deepcopy(module.state_dict())
# any operations on module you want to undo
module.load_state_dict(state)
```

4 Actor-Critic Methods

What is the State Value Estimate Used For? [Slide 34]

- REINFORCE with baseline uses the estimate $\hat{v}(S_t, \mathbf{w})$ of the *first* state of a transition $S_t \to S_{t+1}$ as a *baseline* with which the return G_t from this state is compared.
- Additionally we may use the estimate $\hat{v}(S_{t+1}, \mathbf{w})$ of the *second* state as a bootstrap estimate of G_{t+1}. This obviates the need for full roll-outs and turns our Monte-Carlo (REINFORCE) method into a temporal-difference (***actor-critic***) method, suitable for ***continuing*** tasks.

$$\begin{aligned}
\boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t + \alpha \left(G_t - \hat{v}(S_t, \mathbf{w}) \right) \nabla_{\boldsymbol{\theta}} \log \pi(A_t | S_t, \boldsymbol{\theta}_t) \text{REINFORCE} \\
\boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t + \alpha \left(R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w}) \right) \nabla_{\boldsymbol{\theta}} \log \pi(A_t | S_t, \boldsymbol{\theta}_t) \text{Actor-Critic} \\
&= \boldsymbol{\theta}_t + \alpha \delta_t \nabla_{\boldsymbol{\theta}} \log \pi(A_t | S_t, \boldsymbol{\theta}_t)
\end{aligned}$$

[Sutton and Barto 2018 Sec. 13.5]

Episodic Actor-Critic: The Algorithm [Slide 35]

1. Initialize $\boldsymbol{\theta}$ and state-value weights $\mathbf{w}$, e.g. to $\mathbf{0}$.
2. Loop forever:
   (a) Initialize S, the starting state.
   $I \leftarrow 1$
   (b) Loop while S is not terminal:
      i. Choose $A \sim \pi(\cdot|S, \boldsymbol{\theta})$.
      ii. Take action A, observe R and S'.

$$
\begin{aligned}
\delta &\leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w}) \text{ if } S' \text{ terminal, use 0 for } \hat{v}(S', \boldsymbol{\theta}) \\
\mathbf{w} &\leftarrow \mathbf{w} + \alpha_{\mathbf{w}} \delta \nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w}) \\
\boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \alpha_{\boldsymbol{\theta}} I \delta \nabla_{\boldsymbol{\theta}} \log \pi(A|S, \boldsymbol{\theta}) \\
I &\leftarrow \gamma I \\
S &\leftarrow S'
\end{aligned}
$$

Compare to REINFORCE with Baseline: The Algorithm.

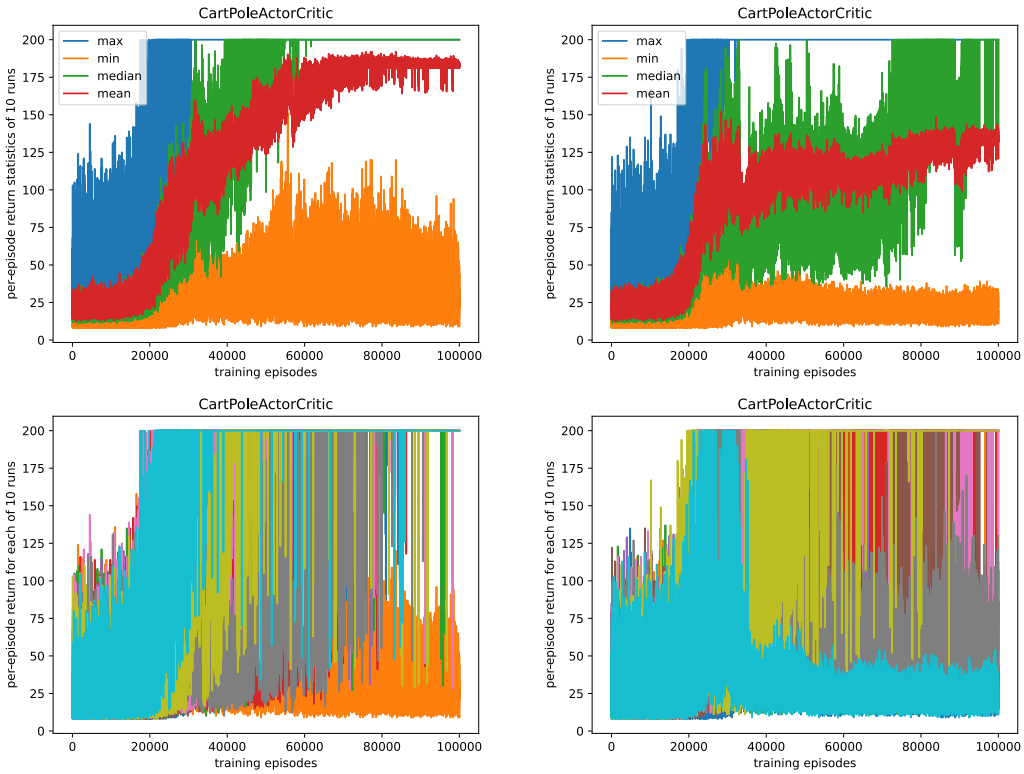
This algorithm uses a state-value critic $\hat{v}(s)$. Alternatively one may use an action-value critic $\hat{q}(s, a)$.

Q: Is this an on-policy or an off-policy algorithm?

A solution is available.

[Slide 36]

One distinct h NN per action | **Single h NN with one output per action**



Parameters:

$\gamma = 1$, $\alpha_{\boldsymbol{\theta}} = 0.00001$, $\alpha_{\mathbf{w}} = 0.0001$

Continuing Actor-Critic: The Algorithm [Slide 37]

1. Initialize $\mathbf{w}$, e.g. to $\mathbf{0}$.
   Initialize $\bar{R}$, e.g. to 0.
   Initialize S, the starting state.
2. Loop forever:
   (a) Choose $A \sim \pi(\cdot|S, \boldsymbol{\theta})$.
   (b) Take action A, observe R and S'.

$$
\begin{aligned}
\delta &\leftarrow R - \bar{R} + \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w}) \\
\bar{R} &\leftarrow \bar{R} + \beta\delta \\
\mathbf{w} &\leftarrow \mathbf{w} + \alpha_{\mathbf{w}}\delta\nabla_{\mathbf{w}}\hat{v}(S, \mathbf{w}) \\
\boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \alpha_{\boldsymbol{\theta}}\delta\nabla_{\boldsymbol{\theta}}\log\pi(A|S, \boldsymbol{\theta}) \\
S &\leftarrow S'
\end{aligned}
$$

Compare to Differential Semi-Gradient Sarsa and Episodic Actor-Critic: The Algorithm.

[Sutton and Barto 2018 Sec. 13.6]

5 Case Studies of Off-Policy Actor-Critic Algorithms

Deterministic Policy Gradient (DPG) [Slide 38]

Instead of *stochastic* policies $\pi(a|s, \boldsymbol{\theta})$ (over discrete or continuous actions spaces) we here consider ***deterministic*** policies $a = \pi(s, \boldsymbol{\theta})$ over ***continuous*** action spaces. This gives rise to the *deterministic* Policy Gradient Theorem:

$$
\begin{aligned}
J(\boldsymbol{\theta}) &= \int_{\mathcal{S}} \mu_\pi(s) q_\pi(s, \pi(s, \boldsymbol{\theta}), \mathbf{w}) \, \mathrm{d}s \\
&= \mathbb{E}_\pi \left[q_\pi(S, \pi(S, \boldsymbol{\theta}), \mathbf{w}) \right] && \text{expectation only over } S\text{, not actions} \\
\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) &= \mathbb{E}_\pi \left[\nabla_a q_\pi(S, a, \mathbf{w}) \nabla_{\boldsymbol{\theta}} \pi(S, \boldsymbol{\theta}) \big|_{a=\pi(S,\boldsymbol{\theta})} \right] \text{chain rule}
\end{aligned}
$$

Note

- The policy gradient theorem for stochastic policies included a sum (integral for continuous action spaces) over all possible actions, and the expectation was accordingly taken over states and actions encountered under π. Here, there is no such sum as only one specific action is deterministically returned by the policy π, and the expectation is only taken over the states encountered under π.

  Thus, $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ can be computed much more efficiently than for stochastic policies. This is the main selling point of DPG.

- $\nabla_{\boldsymbol{\theta}} \pi(S, \boldsymbol{\theta})$ looks similar to our familiar $\nabla_{\boldsymbol{\theta}} \pi(a \mid S, \boldsymbol{\theta})$ but means something quite different: The latter is the gradient of the *probability* (density) of action a (and applies to discrete and continuous action spaces), while the former is the gradient of the *action* a itself (which applies only to continuous action spaces as discrete actions are not differentiable).

  This gives rise to another computational advantage: As the actor becomes increasingly certain, $\pi(a \mid S, \boldsymbol{\theta})$ approaches a singularity around which $\nabla_{\boldsymbol{\theta}} \pi(a \mid S, \boldsymbol{\theta})$ approaches $\pm\infty$, while $\pi(S, \boldsymbol{\theta})$ will generally remain much smoother.

We can now use this with any applicable update rule:

$$
\begin{aligned}
\delta &\leftarrow R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w}) && \text{Sarsa} \\
\delta &\leftarrow R + \gamma \hat{q}(S', \pi(S', \boldsymbol{\theta}), \mathbf{w}) - \hat{q}(S, A, \mathbf{w}) && \text{Q-Learning} \\
\boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \alpha_{\boldsymbol{\theta}} \, \nabla_a \hat{q}(S, a, \mathbf{w}) \nabla_{\boldsymbol{\theta}} \pi(S, \boldsymbol{\theta}) \big|_{a=\pi(S,\boldsymbol{\theta})} \\
\mathbf{w} &\leftarrow \mathbf{w} + \alpha_{\mathbf{w}} \delta \nabla_{\mathbf{w}} \hat{q}(S, A, \mathbf{w})
\end{aligned}
$$

To ensure exploration with a deterministic policy, Q-Learning is a natural choice.

This simplified description hides a lot of technical background. This background is the main interest of DPG; it was not proposed as a practical deep RL algorithm. This is what DDPG is for.

[Silver et al. 2014]

DPG Quick-Thinks [Slide 39]

1. Why does the above Q-Learning rule not include the max operation?

   A solution is available.

2. Isn't Q-Learning the same as Sarsa here, with $A' = \pi(S', \boldsymbol{\theta})$?

   A solution is available.

3. How to implement $\pi(s, \boldsymbol{\theta})$?

   A solution is available.

4. How to compute $\nabla_a \hat{q}(S, a, \mathbf{w})$ using PyTorch? Note that we are taking the derivatives in a, not in the network parameters $\mathbf{w}$.

   A solution is available.

5. DPG is said to be efficient because it avoids the sum/integral over actions. But in the algorithms derived from the policy gradient theorem (e.g., REINFORCE and Actor-Critic) no such sum/integral appears. So this efficiency argument falls apart?

   A solution is available.

Deep Deterministic Policy Gradient (DDPG) [Slide 40]

DDPG = DQN + DPG + *batch normalization*

For exploration in continuous action spaces, DDPG uses a behavior policy $\pi'(s, \boldsymbol{\theta}') = \pi(s, \boldsymbol{\theta}) + \text{noise}$.

Instead of DQN's periodic hard updates $\mathbf{w}' \leftarrow \mathbf{w}$, DDPG uses soft updates (a.k.a. *Polyak averaging*) $\mathbf{w}' \leftarrow \mathbf{w}' + \tau(\mathbf{w} - \mathbf{w}')$ and $\boldsymbol{\theta}' \leftarrow \boldsymbol{\theta}' + \tau(\boldsymbol{\theta} - \boldsymbol{\theta}')$ at every time step.

[Lillicrap et al. 2016]

DDPG: All the Bells and Whistles [Slide 41]

1. Actor-critic with: (DPG)
   - continuous-space actor
   - critic trained by Q-Learning
2. Learns deterministic policies (DPG; DDPG: Polyak-averaged policy parameters)
3. Behavior policy is noisy version of learned policy (DPG)
4. Experience replay (DQN)
5. Separate target network (DQN: periodically frozen; DDPG: soft Polyak updates)

[OpenAI compact explanation]

Twin Delayed DDPG (TD3) [Slide 42]

Common problem with DDPG: *Maximization bias* and/or Diverging Q values (despite the countermeasures inherited from DQN and the soft updates).

Countermeasures:

1. **Clipped Double Q Learning:** Learn *two* Q functions and use the lower prediction for the update targets.
2. **Delayed policy updates:** Update the actor and target networks less frequently than the critic network (say, one actor update for every other critic upate).
3. **Target policy smoothing:** Add noise to the (deterministic) *target* action $\pi(S', \boldsymbol{\theta})$ to counteract spurious actor optimization for critic mispredictions.

   This is different from adding noise to the behavior policy $\pi'(s, \boldsymbol{\theta}')$ for exploration.

[Fujimoto et al. 2018; OpenAI compact explanation]

Maximum-Entropy RL [Slide 43]

Standard, greedy RL policies consider only a tightly-focused action repertoire for a given state: $a = \pi(s) = \operatorname*{argmax}_{a'} q(s, a')$ or $a \sim \pi(a|s) = \mathcal{N}(\cdot | \operatorname*{argmax}_{a'} q(s, a'), \epsilon I)$

Resulting problems:

- Policies trained the same way may differ greatly depending on random seeds etc.
- Behavior can be quite brittle.
  - If the environment changes just a little bit, the policy will not try any other actions.
  - If exploration is not exhaustive (and it de facto never is in policy learning), then small perturbances may totally throw it off.

Remedy: Instead of concentrating on the maximum, define the policy as a Boltzmann distribution with the q function as the negative energy, ensuring positivity, pronounced peaks, and coverage of *all* modes: $\pi(a|s) \propto \exp(q(s, a))$

This defines maximum-entropy RL, and π^* turns out to be of the Boltzmann form:

$$\pi^* = \operatorname*{argmax}_{\pi} \mathop{\mathbb{E}}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(r_t + \alpha H(\pi(\cdot|s_t)) \right) \right]$$

$$H(\pi(\cdot|s)) = -\int \pi(a|s) \log \pi(a|s) \, \mathrm{d}a = \mathop{\mathbb{E}}_{a \sim \pi} [-\log \pi(a|s)]$$

Here, $\tau = (s_0, a_0, r_1, s_1, a_1, \dots)$ denotes a trajectory experienced by the agent.

Here, $H(\tau \sim \pi)$ is the (differential) entropy of the (continuous) random variable π. That is, we want our policy not only to earn high returns but also to be as diverse as possible. The *temperature* parameter α (redundant with the scaling of the rewards) controls the trade-off.

[The equivalence between energy-based (Boltzmann-of-q) models and the maximum-entropy objective is proved by Haarnoja, Tang, et al. 2017 describing the Soft Q-Learning algorithm that was motivated in this way. A high-level overview is provided in a blog post.]

Maximum-Entropy Bellman Equation [Slide 44]

$$\begin{aligned}
v(s) &= \mathop{\mathbb{E}}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(r_t + \alpha H(\pi(\cdot|s_t)) \right) \middle| s_0 = s \right] && \text{cf. MaxEnt } \pi^* \\
q(s, a) &= \mathop{\mathbb{E}}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t + \alpha \sum_{t=1}^{\infty} \gamma^t H(\pi(\cdot|s_t)) \middle| s_0 = s, a_0 = a \right] && \text{no entropy bonus for } a_0 \\
v(s) &= \mathop{\mathbb{E}}_{a \sim \pi} [q(s, a)] + \alpha H(\pi(\cdot|s)) && a_0 \text{ entropy bonus} \\
q(s, a) &= \mathop{\mathbb{E}}_{r, s' \sim \text{env}} [r + \gamma v(s')] && \text{Bellman equation} \\
&= \mathop{\mathbb{E}}_{\substack{r, s' \sim \text{env} \\ a' \sim \pi}} [r + \gamma (q(s', a') + \alpha H(\pi(\cdot|s')))] && \text{with MaxEnt } v(s) \\
&= \mathop{\mathbb{E}}_{\substack{r, s' \sim \text{env} \\ a' \sim \pi}} [r + \gamma (q(s', a') - \alpha \log \pi(a'|s'))] && \text{MaxEnt Bellman eqn.} \\
&\simeq r + \gamma (q(s', a') - \alpha \log \pi(a'|s')) && a' \sim \pi(\cdot|s')
\end{aligned}$$

[Spinning Up]

The final sampling approximation to $q(s, a)$ is a key stepping stone towards practical MaxEnt RL algorithms!

Soft Policy Iteration For Tabular q and π [Slide 45]

Policy Evaluation Iterate with $a' \sim \pi(\cdot|s')$:

$$q(s,a) \leftarrow r + \gamma\left(q(s',a') - \alpha \log \pi(a'|s')\right)$$

This is our sampling approximation of the Maximum-Entropy Bellman Equation.

Policy Improvement Restrict π to a family Π of tractable policies, from which we choose, for each state s, the policy closest to the Boltzmann-of-q policy in terms of the KL divergence $D_{\mathrm{KL}}(\pi\|\rho) = \mathbb{E}_{a\sim\pi}\left[\log \frac{\pi(a)}{\rho(a)}\right]$:

$$\begin{aligned}
\pi \leftarrow & \operatorname*{argmin}_{\pi'\in\Pi} D_{\mathrm{KL}}\left(\pi'(\cdot|s) \,\middle\|\, \frac{1}{Z_{\pi(\cdot|s)}} \exp\left(\frac{1}{\alpha} q(s,\cdot)\right)\right) \\
= & \operatorname*{argmin}_{\pi'\in\Pi} \mathop{\mathbb{E}}_{a\sim\pi'(a|s)}\left[\log \pi'(a|s) - \frac{1}{\alpha} q(s,a)\right] \\
= & \operatorname*{argmax}_{\pi'\in\Pi} \mathop{\mathbb{E}}_{a\sim\pi'(a|s)}\left[q(s,a) - \alpha \log \pi'(a|s)\right] \\
= & \operatorname*{argmax}_{\pi'\in\Pi} q(s,a) \text{ MaxEnt}
\end{aligned}$$

- The normalization constant $Z_{\pi(\cdot|s)}$ has no role in the minimization and can be ignored.
- α is the temperature parameter of the Boltzmann distribution.

Beyond providing the computation of this policy-improvement step, this derivation establishes a close, formal connection between energy-based (Boltzmann-of-q) models and the maximum-entropy objective.

This procedure can fairly easily be proved to converge, under mild conditions, to a globally-optimal policy in the tabular case.

Soft Actor Critic (SAC) [Slide 46]

Adaptation of Soft Policy Iteration For Tabular q and π into a practical Deep-RL algorithm

1. Actor-critic with:
   - continuous-space actor

     It is fairly straightforward to adapt SAC to discrete action spaces. [Christodoulou 2019]
   - critic trained by Q-Learning

     The original version [Haarnoja, Zhou, Abbeel, et al. 2018] additionally included learning a value function to reduce variance during training. However, this was later found not to be necessary and was dropped [Haarnoja, Zhou, Hartikainen, et al. 2019].
2. Learns maximum-entropy policies
3. Experience replay
4. Clipped Double Q Learning (like TD3)
5. Delayed target q network updates (like TD3)

No target policy smoothing is required because the policy is trained to be stochastic anyway.

SAC: Training $\hat{q}_{\mathbf{w}}$ [Slide 47]

From the policy evaluation step:

$$
\begin{aligned}
\hat{q}_{\mathbf{w}}(s,a) &\simeq r + \gamma\left(\hat{q}_{\mathbf{w}}(s',\tilde{a}') - \alpha \log \pi_{\boldsymbol{\theta}}(\tilde{a}'|s')\right) && \tilde{a}' \sim \pi_{\boldsymbol{\theta}}(\cdot|s') \\
L_q(\mathbf{w}_i, \mathcal{D}) &= \mathop{\mathbb{E}}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(\hat{q}_{\mathbf{w}_i}(s,a) - y(r,s')\right)^2\right] && i = 0,1,2 \\
y(r,s') &= r + \gamma\left(\min_{i=1,2} \hat{q}_{\mathbf{w}_i}(s',\tilde{a}') - \alpha \log \pi_{\boldsymbol{\theta}}(\tilde{a}'|s')\right) && \tilde{a}' \sim \pi_{\boldsymbol{\theta}}(\cdot|s')
\end{aligned}
$$

Train $\hat{q}_{\mathbf{w}}(s,a)$ to minimize L, with $\tilde{a}'$ sampled from the current policy, and r and s' drawn from the replay buffer $\mathcal{D}$. [Spinning Up]

Note

- The target networks ($i = 1,2$) are updated less frequently than the critic network ($i = 0$).
- The min operation implements the clipped double Q learning.
- As usual, the bootstrap estimate of the downstream return (the product involving γ) is 0 at the end of an episode.

The optimization of the critic is here formulated as a squared-difference objective. This is mathematically equivalent to our formulation in terms of the TD error δ.

SAC: Training $\pi_{\boldsymbol{\theta}}$ [Slide 48]

From the policy improvement step:

The policy should maximize $v(s) = \mathop{\mathbb{E}}_{a\sim\pi_{\boldsymbol{\theta}}} [q(s,a) - \alpha \log \pi_{\boldsymbol{\theta}}(a|s)]$.

But we cannot easily compute the gradient in $\boldsymbol{\theta}$ of an expectation taken over a random variable a that depends on $\boldsymbol{\theta}$. Thus we *reparametrize* to separate the randomness from $\boldsymbol{\theta}$: [Spinning Up]

$$\begin{aligned}
\tilde{a}_{\boldsymbol{\theta}}(s,\xi) &= \tanh\left(\pi_{\boldsymbol{\theta}}(s) + \sigma_{\boldsymbol{\theta}}(s) \odot \xi\right) && \xi \sim \mathcal{N}(0, I) \\
v(s) &= \mathop{\mathbb{E}}_{\xi\sim\mathcal{N}} \left[q(s, \tilde{a}_{\boldsymbol{\theta}}(s,\xi)) - \alpha \log \pi_{\boldsymbol{\theta}}(\tilde{a}_{\boldsymbol{\theta}}(s,\xi)|s)\right] \\
J_\pi(\boldsymbol{\theta}, \mathcal{D}) &= \mathop{\mathbb{E}}_{\substack{s\sim\mathcal{D} \\ \xi\sim\mathcal{N}}} \left[\min_{j=1,2} \hat{q}_{\bar{\mathbf{w}}_i}(s, \tilde{a}_{\boldsymbol{\theta}}(s,\xi)) - \alpha \log \pi_{\boldsymbol{\theta}}(\tilde{a}_{\boldsymbol{\theta}}(s,\xi)|s)\right] \text{ training objective}
\end{aligned}$$

The same reparametrization trick is used in VAE.

Note

- Here, the squashing function tanh serves to limit the policy to a finite range; this is unrelated to the reparametrization trick.
- J is essentially the same objective function as used by DDPG and TD3 except for the min double Q, the stochasticity of π, and the entropy term.

Quick-Thinks:

1. So what is Π in SAC?
2. Now, can SAC represent multimodal action distributions or not?

A solution is available.

SAC: Auto-Adjusting the Temperature α [Slide 49]

The optimal temperature depends on the reward magnitudes, which depend on the task and on the policy, which changes over time. Moreover, the temperature should be higher in regions where the policy is less certain.

Idea: Fix a minimum average entropy H_0, and optimize α to maximize return:

$$\max_{\pi_0,\ldots,\pi_T} \mathbb{E}\left[\sum_{t=0}^{T} r(s_t, a_t)\right] \text{ s.t. } \forall t, H(\pi_t) \geq H_0$$

Skipping a long, technical derivation involving approximate dynamic programming and constrained optimization by Lagrangian dual gradient descent:

This is achieved by stochastic gradient descent on

$$L(\alpha) = \mathop{\mathbb{E}}_{a_t\sim\pi_t} \left[-\alpha \log \pi_t(a_t|s_t) - \alpha H_0\right].$$

[Haarnoja, Zhou, Hartikainen, et al. 2019]

SAC: The Algorithm [Slide 50]

Algorithm Soft Actor-Critic:

Require: initial q parameters $\mathbf{w}_1$, $\mathbf{w}_2$ and π parameters $\boldsymbol{\theta}$.

Ensure: parameters $\mathbf{w}_1$, $\mathbf{w}_2$, $\boldsymbol{\theta}$ locally maximize empirical return.

$\bar{\mathbf{w}}_i \leftarrow \mathbf{w}_i \qquad i \in \{1, 2\}$

$\mathcal{D} \leftarrow \emptyset$

for each iteration **do**

  for each environment step **do**

    $a_t \sim \pi_{\boldsymbol{\theta}}(a_t \mid s_t)$

    $s_{t+1} \sim p(s_{t+1} \mid s_t, a_t)$

    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r(s_t, a_t), s_{t+1})\}$

  for each gradient step **do**

    $\mathbf{w}_i \leftarrow \mathbf{w}_i - \lambda_q \hat{\nabla}_{\mathbf{w}_i} L_q(\mathbf{w}_i) \;\; i \in \{1, 2\}$

    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \lambda_\pi \hat{\nabla}_{\boldsymbol{\theta}} J_\pi(\boldsymbol{\theta})$

    $\alpha \leftarrow \alpha - \lambda \hat{\nabla}_\alpha L(\alpha)$

    $\bar{\mathbf{w}}_i \leftarrow \tau \mathbf{w}_i + (1 - \tau) \bar{\mathbf{w}}_i \;\; i \in \{1, 2\}$

[Haarnoja, Zhou, Hartikainen, et al. 2019]

6 Offline Reinforcement Learning

Offline RL [Slide 51]

Reliance on online experience can be problematic:

- Online experience may be expensive to collect.
- Exploratory moves may be unsafe or dangerous.
- With function approximators, learning may imply unlearning (which motivates experience replay buffers).

Offline RL (a.k.a. batch RL): fixed dataset only; no interaction

This may be seen as a further step beyond experience replay.

One of the groundbreaking algorithms is Fitted Q Iteration. [Ernst et al. 2005]

Offline training of a neural-network actor and critic:

$$L(\mathbf{w}, \mathcal{D}) = \mathop{\mathbb{E}}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(\hat{q}_{\mathbf{w}}(s, a) - \left(r + \gamma \mathop{\mathbb{E}}_{\tilde{a}' \sim \pi_{\boldsymbol{\theta}}(\cdot \mid s')} [\hat{q}_{\mathbf{w}}(s', \tilde{a}')] \right) \right)^2 \right] \text{critic}$$

$$J(\boldsymbol{\theta}, \mathcal{D}) = \mathop{\mathbb{E}}_{\substack{s \sim \mathcal{D} \\ \tilde{a} \sim \pi_{\boldsymbol{\theta}}(\cdot \mid s)}} [\hat{q}_{\mathbf{w}}(s, \tilde{a})] \qquad \text{actor}$$

Since the learned policy $\pi_{\boldsymbol{\theta}}$ differs from the behavior policy π_β used to generate $\mathcal{D}$, $\hat{q}_{\mathbf{w}}$ is queried for *out-of-distribution* actions chosen by $\pi_{\boldsymbol{\theta}}$ (***distribution shift***). Due to maximization bias their action values will tend to be overestimated.

In standard RL this is eventually corrected by interactions with the environment, but in offline RL there are no such interactions.

Compare L and J to those of the critic and actor of SAC, respectively.

Conservative Q Learning (CQL) [Slide 52]

Idea: Learn a Q function that *bounds* the true Q function from *below*. [Kumar et al. 2020website]

$$L_{\text{CQL}} = L(\mathbf{w}, \mathcal{D}) + \alpha \left(\underbrace{\mathop{\mathbb{E}}_{\substack{s\sim\mathcal{D} \\ \tilde{a}\sim\pi_{\boldsymbol{\theta}}}} [\hat{q}_{\mathbf{w}}(s, \tilde{a})]}_{1.} - \underbrace{\mathop{\mathbb{E}}_{\substack{s\sim\mathcal{D} \\ a\sim\pi_{\beta}}} [\hat{q}_{\mathbf{w}}(s, a)]}_{2.} \right)$$

1. *Penalize* action values by (α times) the (likely over-) estimated, (likely) out-of-distribution action value itself.

   (As usual, $\hat{q}_{\mathbf{w}}$ is trained by stochastic gradient descent, approximating the expectations from samples as indicated.)

   While this obviously solves the overestimation problem for α sufficiently close to 1, this is quite drastic.

2. Tighten this lower bound by *raising* it by the value of a state under the data distribution.

   (To this end, a is drawn from the behavior policy π_{β} that was used to generate $\mathcal{D}$.)

This is a simplified outline of the main idea, glossing over details of the general algorithm. Its theoretical analysis including the proof of the lower bound is not trivial.

In contrast to other offline RL methods, CQL combats distribution shift without any real interactions at all.

CQL can be used with just about any offline RL algorithm. Originally it was demonstrated with QR-DQN and with SAC. [Kumar et al. 2020; superficial blog post]

QR-DQN is a *distributional* variant of DQN that learns the entire Q distribution, not just its expectation. [Dabney et al. 2018]

α is optimized by Lagrangian dual gradient descent (SAC) or by hand (QR-DQN).

7 On-Policy Reinforcement Learning

Parameter Updates vs. Policy Changes [Slide 53]

Observation: A policy update step

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha\delta_t \nabla_{\boldsymbol{\theta}} \log \pi(A_t|S_t, \boldsymbol{\theta}_t)$$

keeps $\boldsymbol{\theta}_{t+1}$ close to $\boldsymbol{\theta}_t$, but this may still incur large changes in $\pi(A|S, \boldsymbol{\theta})$!

This may lead to instabilities and degradation of learning!

Thus, rather than keeping $\boldsymbol{\theta}_{t+1}$ close to $\boldsymbol{\theta}_t$, we may want to keep $\pi(A|S, \boldsymbol{\theta}_{t+1})$ close to $\pi(A|S, \boldsymbol{\theta}_t)$!

Proximal Policy Optimization (PPO) [Slide 54]

How can we take as big as possible an update step without dramatic change to our policy?

PPO updates a policy as

$$\boldsymbol{\theta}_{k+1} = \operatorname*{argmax}_{\boldsymbol{\theta}} \mathop{\mathrm{E}}_{s,a\sim\pi(A|S,\boldsymbol{\theta}_k)}[J(s,a,\boldsymbol{\theta}_k,\boldsymbol{\theta})],$$

where the maximization is *local* and is estimated by several steps of stochastic gradient ascent on $\nabla_\theta J$.

$$\begin{aligned}
&J(s,a,\boldsymbol{\theta}_k,\boldsymbol{\theta}) \\
&= \min\left\{A_{\boldsymbol{\theta}_k}(s,a)\frac{\pi(a|s,\boldsymbol{\theta})}{\pi(a|s,\boldsymbol{\theta}_k)}, A_{\boldsymbol{\theta}_k}(s,a)\mathrm{clip}\left(\frac{\pi(a|s,\boldsymbol{\theta})}{\pi(a|s,\boldsymbol{\theta}_k)}, 1-\epsilon, 1+\epsilon\right)\right\} \\
&= A_{\boldsymbol{\theta}_k}(s,a)\begin{cases} 1+\epsilon & \text{if } A_{\boldsymbol{\theta}_k}(s,a)\geq 0 \text{ and } \pi(a|s,\boldsymbol{\theta})\geq(1+\epsilon)\pi(a|s,\boldsymbol{\theta}_k) \\ 1-\epsilon & \text{if } A_{\boldsymbol{\theta}_k}(s,a)\leq 0 \text{ and } \pi(a|s,\boldsymbol{\theta})\leq(1-\epsilon)\pi(a|s,\boldsymbol{\theta}_k) \\ \dfrac{\pi(a|s,\boldsymbol{\theta})}{\pi(a|s,\boldsymbol{\theta}_k)} & \text{otherwise} \end{cases}
\end{aligned}$$

$A_{\boldsymbol{\theta}_k}(s,a) = q_{\boldsymbol{\theta}_k}(s,a) - v_{\boldsymbol{\theta}_k}(s)$ is the advantage of $\pi(a|s,\boldsymbol{\theta}_k)$.
ϵ controls how far $\pi(a|s,\boldsymbol{\theta}_{k+1})$ is allowed to deviate from $\pi(a|s,\boldsymbol{\theta}_k)$.

The original PPO is a simplified descendant of *Trust Region Policy Optimization* (TRPO) and, like its ancestor, penalizes a KL divergence between the old and new policies instead of the clipping.

[Schulman, Wolski, et al. 2017, Spinning Up]

PPO: The Algorithm [Slide 55]

Algorithm Proximal Policy Optimization:
Require: initial v parameters $\mathbf{w}_0$ and π parameters $\boldsymbol{\theta}_0$.
Ensure: parameters $\mathbf{w}_K$ and $\boldsymbol{\theta}_K$ locally maximize empirical return.
for $k = 0, 1, \dots, K-1$ **do**
Collect trajectories $\mathcal{D}_k = \{\tau_d\}_{d=1}^{D_k}$ via $\pi(a_t|s_t,\boldsymbol{\theta}_k)$ **for** $t = 0, \dots, T_{k,d}$
Compute returns $G(s_t)$ from states s_t
Estimate advantages $A_{\boldsymbol{\theta}_k}(s_t,a_t)$ **for** all t based on $v_{\mathbf{w}_k}$

$$\boldsymbol{\theta}_{k+1} = \operatorname*{argmax}_{\boldsymbol{\theta}} \frac{1}{D_k T_{k,d}} \sum_{d=1}^{D_k}\sum_{t=0}^{T_{k,d}} J(s_t,a_t,\boldsymbol{\theta}_k,\boldsymbol{\theta}) \text{ by SGA}$$

$$\mathbf{w}_{k+t} = \operatorname*{argmin}_{\mathbf{w}} \frac{1}{D_k T_{k,d}} \sum_{d=1}^{D_k}\sum_{t=0}^{T_{k,d}} (v_{\mathbf{w}}(s_t) - G(s_t))^2 \text{ by gradient descent}$$

[Spinning Up]

PPO Quick-Thinks [Slide 56]

1. Why is PPO an on-policy algorithm?
2. Does PPO use a replay buffer?
3. Does PPO work with discrete or continuous policies?
4. Is PPO an actor-critic algorithm?

Generalized Advantage Estimator [Slide 57]

The TD error $\delta_t = r_t + \gamma v(s_{t+1}) - v(s_t)$ is an estimate of the advantage $A(s_t, a_t)$:

$$\begin{aligned}\mathop{\mathrm{E}}_{s_{t+1}}[\delta_t] &= \mathop{\mathrm{E}}_{s_{t+1}}[r_t + \gamma v(s_{t+1}) - v(s_t)] \\ &= \mathop{\mathrm{E}}_{s_{t+1}}[q(s_t, a_t) - v(s_t)] \qquad = A(s_t, a_t)\end{aligned}$$

In a sum of all future δ_t all future $v(s_t)$ cancel, leaving return − baseline:

$$\hat{A}_t^{(\infty)} = \sum_{l=0}^{\infty} \gamma^l \delta_{t+l} = -v(s_t) + \sum_{l=0}^{\infty} \gamma^l r_{t+l} = G_t - v(s_t)$$

Define the GAE as an exponentially-weighted average of these k-step estimators:

$$\begin{aligned}\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} &= (1-\lambda)\sum_{k=1}^{\infty} \lambda^{k-1} \hat{A}_t^{(k)} &&= \sum_{l=0}^{\infty} (\gamma\lambda)^t \delta_{t+l} \\ \hat{A}_t^{\mathrm{GAE}(\gamma,0)} &= \delta_t &&= r_t + \gamma v(s_{t+1}) - v(s_t) \\ \hat{A}_t^{\mathrm{GAE}(\gamma,1)} &= \sum_{l=0}^{\infty} \gamma^l \delta_{t+l} &&= \sum_{l=0}^{\infty} \gamma^l r_{t+l} - v(s_t)\end{aligned}$$

Both γ and λ influence a bias-variance tradeoff when using an approximate v function:

- γ determines the scale of v, which does not depend on λ.

  Taking $\gamma < 1$ will introduce bias into the policy gradient estimate, regardless of the accuracy of v.

- Taking $\lambda < 1$ will introduce bias only when v is inaccurate.

Empirically, choose $\lambda \ll \gamma$ (likely because λ introduces less bias than γ for reasonably-accurate v).

Incidentally, this idea of taking a λ-weighted exponential average of k-step estimators parallels the concept of the *eligibility traces* used by TD(λ). The latter uses it to estimate $v(s)$; here we use it to estimate $A(s, a)$.

[Schulman, Moritz, et al. 2015]

8 Summary

Deep RL Tricks [Slide 58]

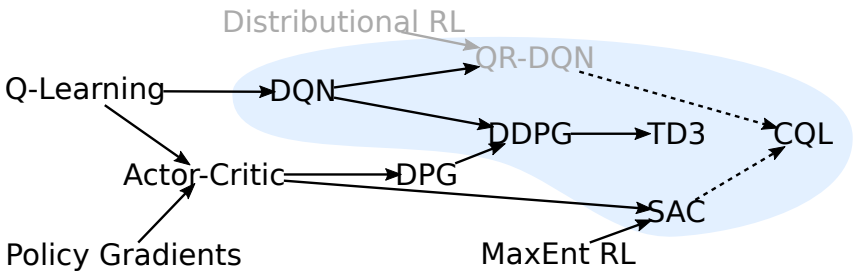
Specific to RL:

- experience replay
- informed exploration
- target network separate from trained network
  - clipped double Q
  - delayed target Q and policy updates, hard or by Polyak averaging
  - target policy smoothing
- …

General Deep Learning:

- batch normalization
- dropout
- gradient clipping
- …

Deep RL Research and Development [Slide 59]



The shaded area covers Deep RL algorithms. Solid arrows point towards an algorithm that builds upon concepts from the arrow tail. Dashed arrows point towards a concept used to enhance the algorithm at the arrow tail.

Of course, this is only a small selection of some representative concepts and algorithms in a hot research area.

- People build on each others' ideas, combine them, discover weaknesses and propose remedies.
- Beyond papers there is high-quality summarizing and comparative documentation such as
  - Lil'Log (personal blog by Lilian Weng, OpenAI)
  - Spinning Up (OpenAI)
- There are high-quality implementations such as
  - Stable Baselines 3 (DLR)
  - Spinning Up (OpenAI)
  - RAY RLlib (Anyscale Inc.)

References [Slide 60]

Christodoulou, Petros (2019). *Soft Actor-Critic for Discrete Action Settings.* arXiv:1910.07207. DOI: 10.48550/arXiv.1910.07207. URL: https://arxiv.org/pdf/1910.07207.pdf.

Dabney, Will, Mark Rowland, Marc Bellemare, and Rémi Munos (2018). "Distributional Reinforcement Learning With Quantile Regression." In: *Proceedings of the AAAI Conference on Artificial Intelligence* 32.

Ernst, Damien, Pierre Geurts, and Louis Wehenkel (2005). "Tree-Based Batch Mode Reinforcement Learning." In: *Journal of Machine Learning Research* 6, pp. 503–556.

Fujimoto, Scott, Herke van Hoof, and David Meger (2018). "Addressing Function Approximation Error in Actor-Critic Methods." In: *Proceedings of the 35th International Conference on Machine Learning.* PMLR, pp. 1587–1596. DOI: 10.48550/arXiv.1802.09477. URL: https://arxiv.org/pdf/1802.09477.pdf.

Haarnoja, Tuomas, Haoran Tang, Pieter Abbeel, and Sergey Levine (2017). "Reinforcement Learning with Deep Energy-Based Policies." In: *Proceedings of the 34th International Conference on Machine Learning.* PMLR, pp. 1352–1361.

Haarnoja, Tuomas, Aurick Zhou, Pieter Abbeel, and Sergey Levine (2018). "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor." In: *Proceedings of the 35th International Conference on Machine Learning*, pp. 1861–1870.

Haarnoja, Tuomas, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine (2019). *Soft Actor-Critic Algorithms and Applications.* arXiv:1812.05905. DOI: 10.48550/arXiv.1812.05905. URL: https://arxiv.org/pdf/1812.05905.pdf.

Kumar, Aviral, Aurick Zhou, George Tucker, and Sergey Levine (2020). "Conservative Q-Learning for Offline Reinforcement Learning." In: *Advances in Neural Information Processing Systems.* Ed. by H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin. Vol. 33. Curran Associates, Inc., pp. 1179–1191.

Lillicrap, Timothy, Jonathan Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra (2016). "Continuous Control with Deep Reinforcement Learning." In: *International Conference on Learning Representations.* DOI: 10.48550/arXiv.1509.02971. URL: https://arxiv.org/pdf/1509.02971.pdf.

Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Andrei Rusu, Joel Veness, Marc Bellemare, Alex Graves, Martin Riedmiller, Andreas Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis (Feb. 2015). "Human-level control through deep reinforcement learning." In: *Nature* 518, pp. 529–533. DOI: 10.1038/nature14236. URL: https://web.stanford.edu/class/psych209/Readings/MnihEtAlHassibis15NatureControlDeepRL.pdf.

Schulman, John, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel (2015). *High-Dimensional Continuous Control Using Generalized Advantage Estimation.* arXiv:1506.02438. DOI: 10.48550/arXiv.1506.02438. URL: http://arxiv.org/abs/1506.02438v6.

Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov (2017). *Proximal Policy Optimization Algorithms.* arXiv:1707.06347. DOI: 10.48550/arXiv.1707.06347. URL: http://arxiv.org/abs/1707.06347v2.

Silver, David, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller (2014). "Deterministic Policy Gradient Algorithms." In: *Proceedings of the 31st International Conference on Machine Learning*, pp. 387–395.

Sutton, Richard and Andrew Barto (2018). *Reinforcement Learning: An Introduction.* Cambridge, MA, USA: MIT Press.